

Học máy: Tối ưu hóa, Thiên lệch-Phương sai, và Điều chuẩn

Quoc Kien

1. Tối ưu hóa bằng Gradient Descent

Bản đồ ký hiệu tối ưu hóa và regularization

Ký hiệu	Nghĩa	Cách dùng
$J(\theta)$	Hàm mục tiêu/loss cần tối thiểu hóa	Theo dõi xem mô hình đang tốt lên hay xấu đi.
θ hoặc β	Tham số mô hình	Thứ được cập nhật qua gradient descent.
α	Learning rate	Bước đi lớn hay nhỏ trên hướng gradient.
∇J	Gradient	Hướng tăng nhanh nhất của loss.
λ	Độ mạnh regularization	Lớn hơn nghĩa là phạt mô hình phức tạp mạnh hơn.
Bias	Sai lệch hệ thống của mô hình trung bình	Cao khi mô hình quá đơn giản.
Variance	Độ nhạy với tập huấn luyện	Cao khi mô hình quá phức tạp.
Train/validation/test	Ba vai trò dữ liệu khác nhau	Học tham số, chọn siêu tham số, đánh giá cuối.

Quy trình khi gặp bài tính: viết loss, tính gradient của phần dữ liệu, cộng gradient của penalty nếu có, cập nhật tham số, rồi giải thích kết quả theo bias/variance hoặc overfit/underfit.

Giả sử ta tối thiểu hóa loss hồi quy tuyến tính:

$$L(\beta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)^2$$

Gradient descent cập nhật tham số theo quy tắc:

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \nabla L(\beta^{(t)})$$

Trong đó α là learning rate. Nếu α quá nhỏ, mô hình học chậm. Nếu α quá lớn, loss có thể dao động hoặc tăng.

Với từng hệ số β_j :

$$\frac{\partial L}{\partial \beta_j} = -\frac{1}{n} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i) x_{i,j}$$

Suy ra cập nhật:

$$\beta_j \leftarrow \beta_j + \frac{\alpha}{n} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i) x_{i,j}$$

Thực tế: nếu residual $y_i - \hat{y}_i$ dương, mô hình đang dự đoán thấp hơn thực tế. Ta tăng các hệ số tương ứng với đặc trưng dương để kéo dự đoán lên.

2. Batch, Stochastic, và Mini-batch

Biến thể	Dữ liệu dùng trong một lần cập nhật	Công thức	Khi nào nhớ tới
Batch Gradient Descent	Toàn bộ n mẫu	$\beta_j \leftarrow \beta_j + \frac{\alpha}{n} \sum_i (y_i - \hat{y}_i) x_{i,j}$	Mượt, ổn định, nhưng chậm với dữ liệu lớn.
Stochastic Gradient Descent	Một mẫu ngẫu nhiên	$\beta_j \leftarrow \beta_j + \alpha (y_i - \hat{y}_i) x_{i,j}$	Nhanh, nhiễu cao, loss dao động.
Mini-batch Gradient Descent	Một nhóm m mẫu	$\beta_j \leftarrow \beta_j + \frac{\alpha}{m} \sum_{i \in \mathcal{B}} (y_i - \hat{y}_i) x_{i,j}$	Cân bằng giữa tốc độ, vector hóa, và ổn định.

```
import numpy as np
import matplotlib.pyplot as plt

beta_grid = np.linspace(-1, 5, 300)
loss_grid = (beta_grid - 2.3) ** 2 + 0.4
```

```

beta = -0.5
alpha = 0.22
path = [beta]
for _ in range(12):
    grad = 2 * (beta - 2.3)
    beta = beta - alpha * grad
    path.append(beta)

path = np.array(path)
path_loss = (path - 2.3) ** 2 + 0.4

plt.figure(figsize=(7, 4))
plt.plot(beta_grid, loss_grid, color="#4C78A8", label="loss")
plt.scatter(path, path_loss, color="#F58518", zorder=3, label="các bước cập
    ↪ nhật")
for a, b, la, lb in zip(path[:-1], path[1:], path_loss[:-1], path_loss[1:]):
    plt.annotate("", xy=(b, lb), xytext=(a, la), arrowprops={"arrowstyle":
    ↪ "->", "color": "#F58518"})
plt.xlabel("tham số beta")
plt.ylabel("loss")
plt.title("Learning rate điều khiển độ dài bước đi")
plt.legend()
plt.grid(alpha=0.25)
plt.show()

```

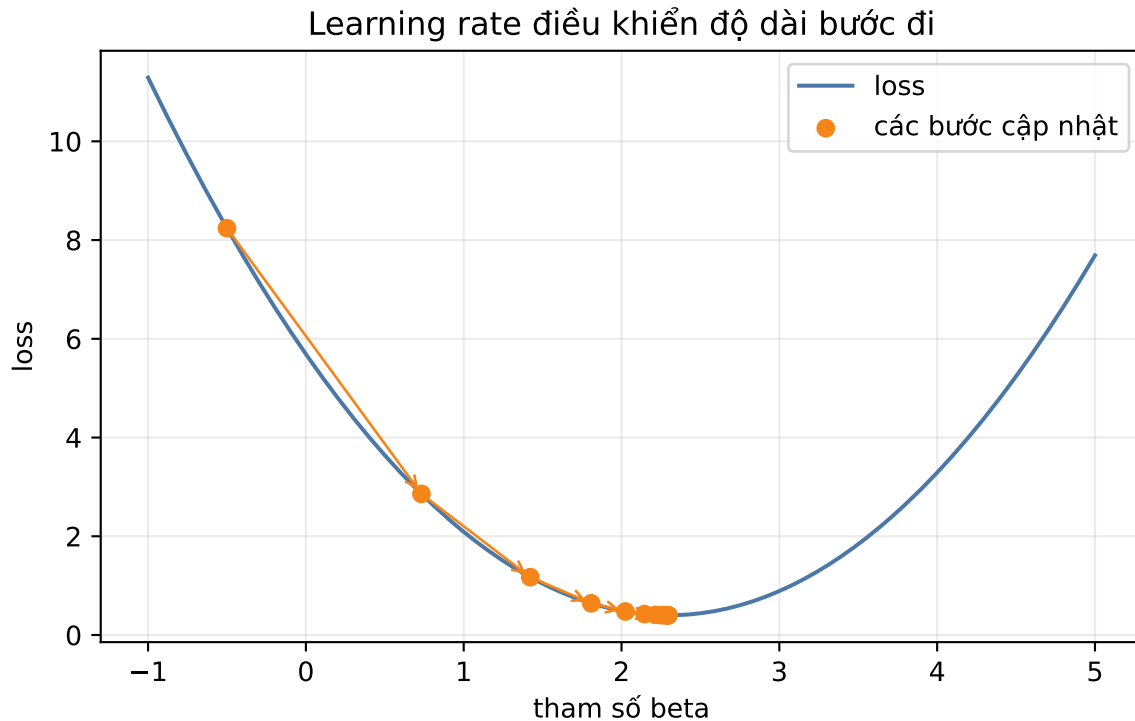


Figure 1: Gradient descent đi từng bước xuống đáy của hàm loss lồi.

3. Underfitting và Overfitting

Khi mô hình gặp dữ liệu mới, ta quan tâm test loss:

$$L_{\text{test}} = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [(y - \hat{f}(\mathbf{x}))^2]$$

Hai kiểu lỗi chính:

- **Underfitting, high bias:** train loss cao, test loss cao. Mô hình quá đơn giản, không học được quy luật thật.
- **Overfitting, high variance:** train loss thấp, test loss cao. Mô hình quá linh hoạt, học cả nhiễu của tập train.

Một câu nhớ nhanh: bias là lỗi do giả định quá cứng; variance là lỗi do mô hình thay đổi quá mạnh khi tập train thay đổi.

4. Bias-Variance Decomposition

Giả sử:

$$y = f(\mathbf{x}) + \epsilon, \quad \mathbb{E}[\epsilon] = 0, \quad \text{Var}(\epsilon) = \sigma^2$$

Mô hình học từ một tập dữ liệu ngẫu nhiên $\mathcal{D}$, nên dự đoán $\hat{f}(\mathbf{x}; \mathcal{D})$ cũng là biến ngẫu nhiên. Tại một điểm $\mathbf{x}$ cố định:

$$\mathbb{E}[(y - \hat{f})^2] = \text{Bias}(\hat{f})^2 + \text{Var}(\hat{f}) + \sigma^2$$

Suy luận từng bước:

$$\mathbb{E}[(y - \hat{f})^2] = \mathbb{E}[(f + \epsilon - \hat{f})^2]$$

Khai triển bình phương:

$$\mathbb{E}[(f - \hat{f})^2 + 2\epsilon(f - \hat{f}) + \epsilon^2]$$

Vì $\mathbb{E}[\epsilon] = 0$, hạng chéo bằng 0. Vì $\mathbb{E}[\epsilon^2] = \sigma^2$, còn:

$$\mathbb{E}_{\mathcal{D}}[(f - \hat{f})^2] + \sigma^2$$

Thêm và bớt dự đoán trung bình $\mathbb{E}_{\mathcal{D}}[\hat{f}]$:

$$f - \hat{f} = (f - \mathbb{E}_{\mathcal{D}}[\hat{f}]) + (\mathbb{E}_{\mathcal{D}}[\hat{f}] - \hat{f})$$

Hạng thứ nhất tạo **bias bình phương**. Hạng thứ hai tạo **variance**. Hạng chéo bằng 0 vì độ lệch quanh trung bình có kỳ vọng bằng 0.

5. Regularization

Regularization thêm hình phạt vào loss để mô hình bớt đuối theo nhiễu:

$$L_{\text{reg}}(\beta) = L(\beta) + \lambda R(\beta)$$

Trong đó $\lambda \geq 0$ điều khiển độ mạnh của phạt.

Phương pháp	Penalty	Tác dụng
Ridge, L_2	$\lambda \sum_j \beta_j^2$	Co nhỏ hệ số một cách mượt, giảm variance.
Lasso, L_1	$\lambda \sum_j \beta_j $	Có thể đẩy một số hệ số về đúng 0, chọn đặc trưng.
Elastic Net	$\lambda_1 \ \beta\ _1 + \lambda_2 \ \beta\ _2^2$	Kết hợp sparse và ổn định.

Nếu λ quá nhỏ, mô hình có thể overfit. Nếu λ quá lớn, mô hình có thể underfit.

6. Chọn siêu tham số bằng Cross-validation

Quy trình K -fold cross-validation:

1. Chia tập train thành K fold.
2. Với mỗi ứng viên λ , train trên $K - 1$ fold và validate trên fold còn lại.
3. Lặp để mỗi fold được làm validation một lần.
4. Lấy trung bình validation loss.
5. Chọn λ có validation loss trung bình nhỏ nhất.
6. Train lại trên toàn bộ tập train bằng λ đã chọn.

Không dùng test set để chọn λ , vì test set phải là phần đánh giá cuối cùng.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(3)
x = np.linspace(-3, 3, 35)
true_y = np.sin(x)
y = true_y + np.random.normal(0, 0.22, size=x.shape)
x_grid = np.linspace(-3, 3, 300)
```

```

true_grid = np.sin(x_grid)

degrees = [1, 5, 18]
titles = ["High bias\nunderfit", "Cân bằng", "High variance\noverfit"]
fig, axes = plt.subplots(1, 3, figsize=(10, 3.4), sharey=True)

for ax, degree, title in zip(axes, degrees, titles):
    coef = np.polyfit(x, y, degree)
    pred = np.polyval(coef, x_grid)
    train_mse = np.mean((np.polyval(coef, x) - y) ** 2)
    test_mse = np.mean((pred - true_grid) ** 2)
    ax.scatter(x, y, color="#4C78A8", s=35, label="mẫu train")
    ax.plot(x_grid, true_grid, color="black", linestyle="--", label="hàm
    ⇨ thật")
    ax.plot(x_grid, pred, color="#F58518", linewidth=2, label="mô hình")
    ax.set_title(f"{title}\nbậc={degree}")
    ax.text(-2.9, 1.15, f"train MSE={train_mse:.2f}\ntest
    ⇨ MSE={test_mse:.2f}", fontsize=8)
    ax.set_ylim(-1.5, 1.5)
    ax.grid(alpha=0.25)
axes[0].legend(fontsize=8, loc="lower left")
plt.tight_layout()
plt.show()

```

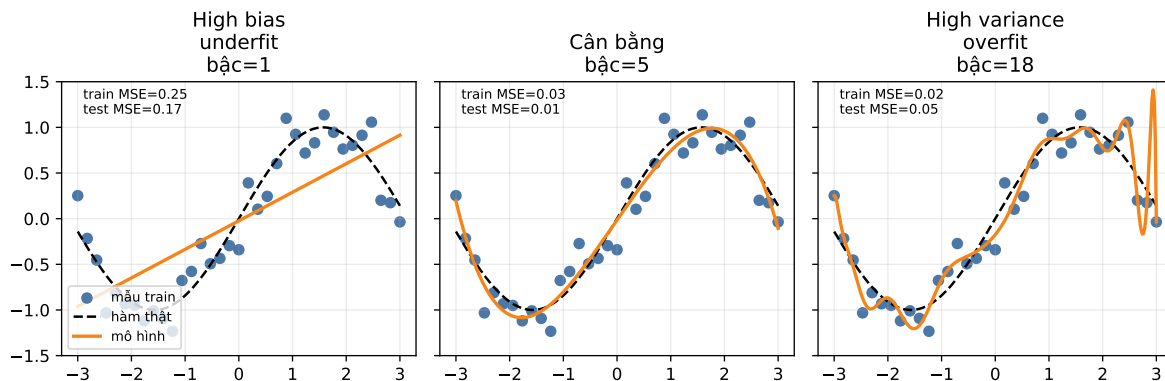


Figure 2: Underfitting, fit hợp lý, và overfitting trên cùng một bộ dữ liệu nhiễu.

7. Quy trình xử lý bài tối ưu hóa và regularization

Khi đề yêu cầu một bước cập nhật:

1. Viết dự đoán hiện tại tại $\hat{y}$.
2. Viết residual hoặc sai số dự đoán.
3. Tính gradient phần loss dữ liệu.
4. Tính gradient penalty:

$$\nabla_{\beta} \lambda \|\beta\|_2^2 = 2\lambda\beta$$

5. Cộng gradient:

$$\nabla J_{total} = \nabla J_{data} + \nabla J_{penalty}$$

6. Cập nhật:

$$\beta_{new} = \beta - \alpha \nabla J_{total}$$

Bẫy thường gặp: regularization thường không phạt bias/intercept trong nhiều triển khai. Nếu đề không nói rõ, hãy nêu giả định trước khi tính.

8. Bảng Công thức Thi: Bias, Variance, Regularization

Dấu hiệu	Chẩn đoán	Cách xử lý
Train loss cao, test loss cao	High bias	Thêm đặc trưng, tăng bậc mô hình, giảm regularization.
Train loss thấp, test loss cao	High variance	Tăng regularization, đơn giản hóa mô hình, thêm dữ liệu.
Train loss thấp, test loss thấp	Fit tốt	Giữ mô hình, kiểm tra thêm bằng dữ liệu mới.
Validation loss nhỏ nhất tại λ trung bình	Trade-off tốt	Chọn λ đó rồi train lại trên full train.
Loss dao động mạnh khi train	Learning rate lớn hoặc SGD nhiều	Giảm α , dùng mini-batch, chuẩn hóa dữ liệu.

Gradient cần nhớ:

$$\nabla L = -\frac{1}{n} \mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta)$$

Ridge gradient add-on:

$$\nabla L_{\text{ridge}} = \nabla L + 2\lambda\beta$$

Bias-variance:

$$\text{MSE} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Dạng bài: một bước gradient descent có regularization

Giả sử loss Ridge:

$$J(\beta) = \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2$$

Gradient tổng:

$$\nabla J = -\frac{1}{n} \mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta) + 2\lambda\beta$$

Nếu không phạt bias, tách $\beta = (\beta_0, \beta_1, \dots, \beta_p)$ và chỉ phạt từ β_1 trở đi:

$$\nabla_{\beta_0} J = \nabla_{\beta_0} J_{\text{data}}$$

$$\nabla_{\beta_j} J = \nabla_{\beta_j} J_{\text{data}} + 2\lambda\beta_j, \quad j \geq 1$$

Sau đó cập nhật:

$$\beta_{\text{new}} = \beta - \alpha \nabla J$$

Cách kiểm tra trực giác: nếu λ lớn, các hệ số không phải bias bị kéo về 0 mạnh hơn. Nếu bắt đầu tại $\beta = 0$, gradient của penalty bằng 0 ở bước đầu tiên; tác dụng regularization xuất hiện rõ hơn ở các bước sau.

9. Bộ bài tập mẫu có lời giải

Bài 1: Tính objective Ridge

Đề bài. Một mô hình có train loss 0.20 và trọng số $\beta = (2, -1, 0.5)$. Tính ridge objective với $\lambda = 0.1$.

Lời giải.

Tính penalty:

$$\|\beta\|_2^2 = 2^2 + (-1)^2 + 0.5^2 = 4 + 1 + 0.25 = 5.25$$

Nhân với λ :

$$\lambda\|\beta\|_2^2 = 0.1(5.25) = 0.525$$

Cộng với loss gốc:

$$L_{\text{reg}} = 0.20 + 0.525 = 0.725$$

Kết luận. Ridge objective bằng 0.725.

Bài 2: Chọn λ từ bảng cross-validation

Đề bài. Sau 5-fold cross-validation, ta có validation MSE:

λ	fold 1	fold 2	fold 3	fold 4	fold 5
0.00	0.92	1.10	0.98	1.05	1.20
0.01	0.70	0.78	0.74	0.76	0.81
0.10	0.54	0.57	0.55	0.58	0.56
1.00	0.61	0.65	0.64	0.62	0.67
10.00	1.20	1.15	1.25	1.18	1.22

Chọn λ tốt nhất.

Lời giải. Tính trung bình từng dòng:

$$\lambda = 0 : \frac{0.92 + 1.10 + 0.98 + 1.05 + 1.20}{5} = 1.050$$

$$\lambda = 0.01 : \frac{0.70 + 0.78 + 0.74 + 0.76 + 0.81}{5} = 0.758$$

$$\lambda = 0.10 : \frac{0.54 + 0.57 + 0.55 + 0.58 + 0.56}{5} = 0.560$$

$$\lambda = 1.00 : \frac{0.61 + 0.65 + 0.64 + 0.62 + 0.67}{5} = 0.638$$

$$\lambda = 10.00 : \frac{1.20 + 1.15 + 1.25 + 1.18 + 1.22}{5} = 1.200$$

Bảng tóm tắt:

λ	validation MSE trung bình
0.00	1.050
0.01	0.758
0.10	0.560
1.00	0.638
10.00	1.200

Kết luận. Chọn $\lambda = 0.10$ vì validation MSE trung bình nhỏ nhất. $\lambda = 0$ có dấu hiệu overfit, còn $\lambda = 10$ quá mạnh nên dễ underfit.

Bài 3: Một bước Gradient Descent trên dữ liệu 12 dòng

Đề bài. Dữ liệu có một đặc trưng x và intercept. Bắt đầu với $\beta_0 = 0, \beta_1 = 0$, learning rate $\alpha = 0.01$. Tính một bước batch gradient descent cho loss MSE.

dòng	x	y
1	1	4
2	2	5
3	3	7
4	4	8
5	5	11
6	6	13
7	7	15
8	8	16
9	9	19

dòng	x	y
10	10	21
11	11	22
12	12	25

Lời giải. Với $\beta_0 = \beta_1 = 0$, mọi dự đoán ban đầu bằng 0, nên residual $r_i = y_i$.

Gradient theo dạng cập nhật cộng:

$$\beta_j \leftarrow \beta_j + \frac{\alpha}{n} \sum_{i=1}^n r_i x_{i,j}$$

Với intercept, $x_{i,0} = 1$.

Tính tổng residual:

$$\sum r_i = 4 + 5 + 7 + 8 + 11 + 13 + 15 + 16 + 19 + 21 + 22 + 25 = 166$$

Tính tổng residual nhân đặc trưng:

$$\sum r_i x_i = 4(1) + 5(2) + 7(3) + 8(4) + 11(5) + 13(6) + 15(7) + 16(8) + 19(9) + 21(10) + 22(11) + 25(12) = 1356$$

Cập nhật intercept:

$$\beta_0^{new} = 0 + \frac{0.01}{12}(166) \approx 0.1383$$

Cập nhật slope:

$$\beta_1^{new} = 0 + \frac{0.01}{12}(1356) = 1.1300$$

Kết luận. Một bước cập nhật đã kéo đường dự đoán từ đường phẳng $\hat{y} = 0$ về phía dữ liệu. Trong bài thi, chỉ cần nhớ: residual nhân đặc trưng rồi lấy trung bình.